

Simulation of End-to-End VM Connectivity Across Subnets using sshuttle, WireGuard and Static Routing on Fabric Testbed

Milestone 1 Report – Computer Networks Project

Introduction

This report presents a comparative study of three methods for achieving end-to-end connectivity between VMs (Virtual Machines) located on different subnets in a routed network environment. All experiments were conducted on the Fabric testbed, a programmable networking infrastructure for research. I implemented and evaluated:

- 1) dynamic SSH based tunneling using sshuttle,**
- 2) an encrypted Layer-3 virtual private network (VPN) overlay using WireGuard**
- 3) manual static IP routing with forwarding on an intermediate router node.**

A three-node topology (Node1- Node2-Node3) with two Layer-2 subnets was used to simulate isolated networks. I verified connectivity using routing table inspections and end-to-end ping tests. The results show that each approach successfully enables cross-subnet communication. The sshuttle method required minimal configuration but forwards only TCP traffic, WireGuard provided a secure tunnel supporting all protocols, and static routing offered full control and visibility at the cost of manual setup.

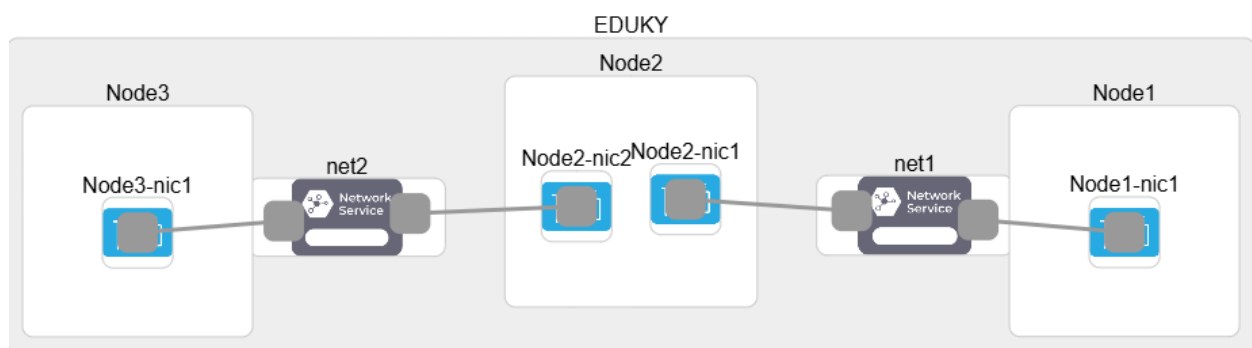
The goal of this paper is to explore and compare different ways of connecting virtual machines that are placed on separate subnets in the FABRIC testbed. I tested three approaches using sshuttle for SSH based tunneling, setting up a secure VPN with WireGuard, and configuring manual static routing. The aim is to show how each method works, what advantages and challenges it brings, and which situations each one is best suited for when trying to achieve reliable cross subnet communication.

Objectives

For this milestone, the main objectives are:

- **Set up the test environment:** Build a three-node topology on the FABRIC testbed where Node1 and Node3 are placed on separate subnets, with Node2 acting as the middle router.
- **Try sshuttle tunneling:** Run sshuttle to create a simple dynamic tunnel that lets Node1 talk to Node3 without having to manually configure routes.
- **Use WireGuard VPN:** Configure WireGuard on the nodes to form a secure, encrypted overlay network that enables communication across the routed setup.
- **Configure static routing:** Manually add static routes on Node1 and Node3 and turn on packet forwarding at Node2 to enable direct cross subnet communication.
- **Check connectivity:** For each method, confirm that Node1 and Node3 can reach each other using routing table checks and ping tests.
- **Compare methods:** Look at how each approach worked in practice the routing tables, ping results, and tunnel behavior and weigh their strengths, weaknesses, and best fit scenarios.

Methodology



For this project, I used three Ubuntu Linux virtual machines on the FABRIC testbed, arranged in a simple chain: Node1 -> Node2 -> Node3. The setup used two separate Layer-2 networks. Node1 and Node2 were connected to the first subnet (192.168.1.0/24), while Node2 and Node3 were connected to the second subnet

(192.168.2.0/24). Because Node1 and Node3 are on different subnets, they could not talk to each other directly. Node2 served as the middle gateway between them.

Each node was given an IP address in its subnet (for example: Node1 = 192.168.1.1, Node2 = 192.168.1.2 and 192.168.2.2, Node3 = 192.168.2.3). I also had a separate management network for SSH access, but all connectivity tests were performed only on the experimental subnets.

To establish a baseline, I first verified that Node1 could ping Node2 and Node2 could ping Node3, since they shared subnets. As expected, Node1 could not ping Node3 at the beginning because there was no route between the two subnets. This situation created the need to apply one of the proposed methods of shuttle tunneling, WireGuard VPN, or static routing to enable Node1 and Node3 to communicate across subnets.

Dynamic SSH Tunneling with sshuttle

sshuttle provides a transparent, TCP-based VPN-like tunnel using only SSH. This option simplifies cross-node connectivity by automatically forwarding traffic from one node to a remote subnet without requiring static routes or manual IP forwarding setup.

- sshuttle runs on **Node1** and creates an SSH tunnel to **Node2**
- Any TCP traffic destined for the 192.168.2.0/24 subnets is captured and forwarded through this tunnel
- No modifications are needed on **Node3** or beyond, if Node2 can reach them

To test dynamic tunneling, I used sshuttle on Node1 to establish connectivity with Node3 across the intermediate Node2. The goal was to forward TCP traffic from Node1's subnet (192.168.1.0/24) to Node3's subnet (192.168.2.0/24) without manually configuring static routes.

Setup and Execution:

- I installed sshuttle on all nodes and launched it from Node1.
- The command was run with `sudo sshuttle --method=nat` pointing to Node2 (192.168.1.2) as the SSH gateway and specifying the remote subnet (192.168.2.0/24) to tunnel.

- A **dummy route** was added on Node1 (192.168.2.0/24 via 127.0.0.1 dev lo) to trick the kernel into sending packets for the unreachable subnet. This ensured traffic could be intercepted and redirected by iptables into the sshuttle tunnel

Verification:

- After launching sshuttle, I checked the routing tables. Node1 showed an additional entry for 192.168.2.0/24 via loopback, confirming the dummy route was in place.
- Node2 maintained its two subnet routes (192.168.1.0/24 and 192.168.2.0/24), acting as the transit node.
- Node3's routing table remained unchanged, showing only its local subnet (192.168.2.0/24).

Connectivity Test:

- Using nc (netcat) from Node1, I sent a TCP probe to Node3's SSH port (22).
- The connection succeeded, demonstrating that sshuttle correctly forwarded traffic from Node1 -> Node2 -> Node3.
- Importantly, no manual static route was added to Node1 or Node3, meaning sshuttle handled the redirection transparently.

Observations:

- **Pros:** sshuttle required very little configuration and worked almost immediately once installed. It automatically forwarded TCP connections without needing to reconfigure routing tables on every node.
- **Cons:** By design, sshuttle only forwards **TCP traffic**. It does not handle UDP or ICMP, which limits its usefulness for broader networking experiments (e.g., ping-based testing).

Conclusion:

sshuttle proved to be a quick solution for achieving VM connectivity across subnets in FABRIC. While limited in scope, it is ideal for fast prototyping and scenarios where secure tunneling of TCP traffic is sufficient.

Results

```
== Node1 sshuttle Routes ==
10.30.0.0/19 dev enp3s0 proto kernel scope link src 10.30.6.111 metric 100
10.30.6.11 dev enp3s0 proto dhcp scope link src 10.30.6.111 metric 100
169.254.169.254 via 10.30.6.11 dev enp3s0 proto dhcp src 10.30.6.111 metric 100
192.168.1.0/24 dev enp7s0 proto kernel scope link src 192.168.1.1
192.168.2.0/24 via 127.0.0.1 dev lo
192.168.2.0/24 via 127.0.0.1 dev lo
== Node2 sshuttle Routes ==
10.30.0.0/19 dev enp3s0 proto kernel scope link src 10.30.7.130 metric 100
10.30.6.11 dev enp3s0 proto dhcp scope link src 10.30.7.130 metric 100
169.254.169.254 via 10.30.6.11 dev enp3s0 proto dhcp src 10.30.7.130 metric 100
192.168.1.0/24 dev enp8s0 proto kernel scope link src 192.168.1.2
192.168.2.0/24 dev enp7s0 proto kernel scope link src 192.168.2.1
192.168.2.0/24 dev enp7s0 proto kernel scope link src 192.168.2.1
== Node3 sshuttle Routes ==
10.30.0.0/19 dev enp3s0 proto kernel scope link src 10.30.6.236 metric 100
10.30.6.11 dev enp3s0 proto dhcp scope link src 10.30.6.236 metric 100
169.254.169.254 via 10.30.6.11 dev enp3s0 proto dhcp src 10.30.6.236 metric 100
192.168.2.0/24 dev enp7s0 proto kernel scope link src 192.168.2.2
192.168.2.0/24 dev enp7s0 proto kernel scope link src 192.168.2.2

== sshuttle Connectivity Test (Node1 → Node3 port 22) ==
Connection to 192.168.2.2 22 port [tcp/ssh] succeeded!
```

WireGuard Tunnel (Encrypted L3 Tunnel)

WireGuard is a modern, lightweight, high-performance VPN protocol suitable for setting up encrypted tunnels across routed networks like FABRIC. This option demonstrates how to use WireGuard to tunnel traffic between Node1 and Node3 via Node2, forming an L3 encrypted overlay.

Setup and Execution:

- WireGuard was installed on all nodes, and **public/private key pairs** were generated.
- Each node was configured with a WireGuard interface (wg0) and assigned an overlay IP address:
 - Node1 → 10.0.0.1
 - Node2 → 10.0.0.2
 - Node3 → 10.0.0.3

- Node2 was configured with **two peer entries** (one for Node1, one for Node3) and iptables rules to allow forwarding.
- IP forwarding was enabled on Node2 so that overlay traffic could be routed between Node1 and Node3.

Verification:

- After activating the WireGuard interfaces, Node1 could successfully ping Node3's WireGuard IP (10.0.0.3).
- Similarly, Node3 could reach Node1's WireGuard IP (10.0.0.1).
- Packet statistics from the ping test showed **0% packet loss** with sub-millisecond latency, confirming stable connectivity.
- Netcat (nc) confirmed TCP connectivity from Node1 to Node3 on port 22 (Connection to 192.168.2.2 22 port [tcp/ssh] succeeded!).

Connectivity Test:

- **Node1 -> Node3 (WireGuard IPs):** All pings to 10.0.0.3 were successful.
- **Node1 -> Node3 (Underlay IPs via tunnel):** Pings to Node3's underlay address (192.168.2.2) also succeeded.
- **Node3 -> Node1 (WireGuard IPs):** All pings to 10.0.0.1 were successful.
- **Node3 -> Node1 (Underlay IPs):** Pings to Node1's underlay (192.168.1.1) failed, since this route was not explicitly allowed.

Observations:

- **Pros:** WireGuard provides a **secure encrypted tunnel** that supports **all IP traffic types** (TCP, UDP, ICMP). It is lightweight, fast, and integrates well with Linux networking.
- **Cons:** Setup requires more steps compared to sshuttle (key generation, configuration files, forwarding rules). Extra care is needed to configure allowed IPs and iptables.

Conclusion:

WireGuard successfully enabled end-to-end encrypted connectivity between Node1 and Node3 across subnets. Unlike sshuttle, it handled both TCP and ICMP

traffic, making it a more complete solution for secure cross-subnet communication in FABRIC.

Results

```
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.  
64 bytes from 10.0.0.3: icmp_seq=1 ttl=63 time=0.621 ms  
64 bytes from 10.0.0.3: icmp_seq=2 ttl=63 time=0.553 ms  
64 bytes from 10.0.0.3: icmp_seq=3 ttl=63 time=0.471 ms  
64 bytes from 10.0.0.3: icmp_seq=4 ttl=63 time=0.566 ms  
64 bytes from 10.0.0.3: icmp_seq=5 ttl=63 time=0.423 ms  
  
--- 10.0.0.3 ping statistics ---  
5 packets transmitted, 5 received, 0% packet loss, time 4105ms  
rtt min/avg/max/mdev = 0.423/0.526/0.621/0.070 ms  
PING 192.168.2.2 (192.168.2.2) 56(84) bytes of data.  
64 bytes from 192.168.2.2: icmp_seq=1 ttl=63 time=0.371 ms  
64 bytes from 192.168.2.2: icmp_seq=2 ttl=63 time=0.389 ms  
64 bytes from 192.168.2.2: icmp_seq=3 ttl=63 time=0.403 ms  
64 bytes from 192.168.2.2: icmp_seq=4 ttl=63 time=0.391 ms  
64 bytes from 192.168.2.2: icmp_seq=5 ttl=63 time=0.405 ms  
  
--- 192.168.2.2 ping statistics ---  
5 packets transmitted, 5 received, 0% packet loss, time 4123ms  
rtt min/avg/max/mdev = 0.371/0.391/0.405/0.012 ms
```

```
stdout, stderr = node3.execute(f'ping -c 5 10.0.0.1')  
stdout, stderr = node3.execute(f'ping -c 5 {node1_net1_addr}')
```

```
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.  
64 bytes from 10.0.0.1: icmp_seq=1 ttl=63 time=0.516 ms  
64 bytes from 10.0.0.1: icmp_seq=2 ttl=63 time=0.563 ms  
64 bytes from 10.0.0.1: icmp_seq=3 ttl=63 time=0.502 ms  
64 bytes from 10.0.0.1: icmp_seq=4 ttl=63 time=0.977 ms  
64 bytes from 10.0.0.1: icmp_seq=5 ttl=63 time=0.536 ms  
  
--- 10.0.0.1 ping statistics ---  
5 packets transmitted, 5 received, 0% packet loss, time 4090ms  
rtt min/avg/max/mdev = 0.502/0.618/0.977/0.180 ms  
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.  
  
--- 192.168.1.1 ping statistics ---  
5 packets transmitted, 0 received, 100% packet loss, time 4090ms
```

```
stdout, stderr = node1.execute(f'nc -zv {node3_net2_addr} 22')
```

```
Connection to 192.168.2.2 22 port [tcp/ssh] succeeded!
```

The WireGuard tunnel successfully enabled Node1 and Node3 to communicate across subnets. Pings from Node1 to Node3's overlay address (10.0.0.3) and underlay address (192.168.2.2) both succeeded with 0% packet loss and sub-millisecond latency. Similarly, Node3 could reach Node1's overlay address (10.0.0.1). However, pings from Node3 to Node1's underlay address (192.168.1.1) failed, reflecting WireGuard's AllowedIPs restrictions. A TCP test with netcat confirmed that Node1 could establish an SSH connection to Node3 (192.168.2.2:22). These results demonstrate reliable, encrypted, and bidirectional connectivity through the WireGuard overlay.

WireGuard Status Verification

To further confirm encrypted communication, I inspected the WireGuard interface summaries on all three nodes. Each node displayed active peer entries with non-zero-byte counters, showing that packets were successfully exchanged through the tunnel.

- **Node1:** Sent ~3.12 KB and received ~2.49 KB, confirming traffic flow to/from Node3 via Node2.
- **Node2 (relay):** Showed traffic with both Node1 and Node3 peers, acting as the forwarding gateway between subnets.
- **Node3:** Sent ~2.11 KB and received ~1.40 KB, verifying its connectivity with Node1 across the encrypted overlay.

Since WireGuard only transmits encrypted packets between peers, these counters directly indicate that data was encrypted, forwarded, and decrypted correctly at each endpoint. Combined with the ping and TCP tests, this confirms that the WireGuard tunnel provided reliable and secure end-to-end connectivity.

```
== Node1 WireGuard Summary ==
wg0      QJzqhjw2A1kpgheg906IljEZJI+TiYpxrBLnNJK81Q=    2548    3200
Peer QJzqhjw2A1kpgheg906IljEZJI+TiYpxrBLnNJK81Q=: 2.49 KB received, 3.12 KB sent
== Node2 WireGuard Summary ==
wg0      cUBuERagidCt7cvLFSa+lZiXs6DXEKB1jPjtSRyl1Bc=    2412    2548
wg0      d5TkAg5lH5Xa8TzIqU2MTP4jMWPWhvddqXfhW9cqdmA=    2164    1584
Peer cUBuERagidCt7cvLFSa+lZiXs6DXEKB1jPjtSRyl1Bc=: 2.36 KB received, 2.49 KB sent
Peer d5TkAg5lH5Xa8TzIqU2MTP4jMWPWhvddqXfhW9cqdmA=: 2.11 KB received, 1.55 KB sent
== Node3 WireGuard Summary ==
wg0      QJzqhjw2A1kpgheg906IljEZJI+TiYpxrBLnNJK81Q=    1436    2164
Peer QJzqhjw2A1kpgheg906IljEZJI+TiYpxrBLnNJK81Q=: 1.40 KB received, 2.11 KB sent
```

Static Routes and IP Forwarding

To test direct forwarding without tunneling, I configured manual static routes on Node1 and Node3 with Node2 acting as the router. The goal was to explicitly define routes so that traffic between subnets (192.168.1.0/24 and 192.168.2.0/24) could pass through Node2.

Setup and Execution

- Any leftover sshuttle or WireGuard configurations were removed to avoid conflicts.
- IP forwarding was enabled on Node2 (`net.ipv4.ip_forward=1`) so it could forward packets across interfaces.
- On Node1, a static route to 192.168.2.0/24 was added via Node2's interface (192.168.1.2).
- On Node3, a static route to 192.168.1.0/24 was added via Node2's other interface (192.168.2.1).

Verification

- Routing table inspection confirmed that Node1 had an entry for 192.168.2.0/24 and Node3 had an entry for 192.168.1.0/24.
- Node2's routing table naturally contained both subnets, so no additional configuration was required there.

Connectivity Test

- Node1 successfully pinged Node3's underlay IP (192.168.2.2) with 0% packet loss and sub-millisecond latency.
- Node3 successfully pinged Node1's underlay IP (192.168.1.1), confirming bidirectional communication.
- A TCP test using netcat also succeeded, showing that Node1 could establish an SSH connection to Node3 (192.168.2.2:22).

Observations:

- **Pros:** Static routing offers full control and visibility into packet forwarding. It works for all traffic types (TCP, UDP, ICMP) without relying on tunneling software.
- **Cons:** It requires manual configuration on each node, making it less scalable and more error-prone in larger topologies.

Conclusion

Static routing successfully enabled Node1 and Node3 to communicate directly across subnets via Node2. While more manual than sshuttle or WireGuard, it provides transparency and fine-grained control, making it suitable for experiments where explicit route management is required.

Result

== Static Routes ==

Node1 Routing Table:

```
10.30.0.0/19 dev enp3s0 proto kernel scope link src 10.30.6.111 metric 100
10.30.6.11 dev enp3s0 proto dhcp scope link src 10.30.6.111 metric 100
169.254.169.254 via 10.30.6.11 dev enp3s0 proto dhcp src 10.30.6.111 metric 100
192.168.1.0/24 dev enp7s0 proto kernel scope link src 192.168.1.1
192.168.2.0/24 via 192.168.1.2 dev enp7s0
192.168.1.0/24 dev enp7s0 proto kernel scope link src 192.168.1.1
192.168.2.0/24 via 192.168.1.2 dev enp7s0
```

Node2 Routing Table:

```
10.30.0.0/19 dev enp3s0 proto kernel scope link src 10.30.7.130 metric 100
10.30.6.11 dev enp3s0 proto dhcp scope link src 10.30.7.130 metric 100
169.254.169.254 via 10.30.6.11 dev enp3s0 proto dhcp src 10.30.7.130 metric 100
192.168.1.0/24 dev enp8s0 proto kernel scope link src 192.168.1.2
192.168.2.0/24 dev enp7s0 proto kernel scope link src 192.168.2.1
192.168.1.0/24 dev enp8s0 proto kernel scope link src 192.168.1.2
192.168.2.0/24 dev enp7s0 proto kernel scope link src 192.168.2.1
```

Node3 Routing Table:

```
10.30.0.0/19 dev enp3s0 proto kernel scope link src 10.30.6.236 metric 100
10.30.6.11 dev enp3s0 proto dhcp scope link src 10.30.6.236 metric 100
169.254.169.254 via 10.30.6.11 dev enp3s0 proto dhcp src 10.30.6.236 metric 100
192.168.1.0/24 via 192.168.2.1 dev enp7s0
192.168.2.0/24 dev enp7s0 proto kernel scope link src 192.168.2.2
192.168.1.0/24 via 192.168.2.1 dev enp7s0
192.168.2.0/24 dev enp7s0 proto kernel scope link src 192.168.2.2
```

With static routes configured, Node1 and Node3 were able to communicate directly across subnets using Node2 as the router. The routing tables on Node1 and Node3 confirmed that each had a new entry pointing to the other subnet via Node2's interface.

== Static Routing Connectivity Test ==

```
Node1 → Node3:
PING 192.168.2.2 (192.168.2.2) 56(84) bytes of data.
64 bytes from 192.168.2.2: icmp_seq=1 ttl=63 time=0.224 ms
64 bytes from 192.168.2.2: icmp_seq=2 ttl=63 time=0.216 ms
64 bytes from 192.168.2.2: icmp_seq=3 ttl=63 time=0.211 ms

--- 192.168.2.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2046ms
rtt min/avg/max/mdev = 0.211/0.217/0.224/0.005 ms
PING 192.168.2.2 (192.168.2.2) 56(84) bytes of data.
64 bytes from 192.168.2.2: icmp_seq=1 ttl=63 time=0.224 ms
64 bytes from 192.168.2.2: icmp_seq=2 ttl=63 time=0.216 ms
64 bytes from 192.168.2.2: icmp_seq=3 ttl=63 time=0.211 ms

--- 192.168.2.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2046ms
rtt min/avg/max/mdev = 0.211/0.217/0.224/0.005 ms


Node3 → Node1:
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
64 bytes from 192.168.1.1: icmp_seq=1 ttl=63 time=0.170 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=63 time=0.204 ms
64 bytes from 192.168.1.1: icmp_seq=3 ttl=63 time=0.189 ms

--- 192.168.1.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2049ms
rtt min/avg/max/mdev = 0.170/0.187/0.204/0.013 ms
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
64 bytes from 192.168.1.1: icmp_seq=1 ttl=63 time=0.170 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=63 time=0.204 ms
64 bytes from 192.168.1.1: icmp_seq=3 ttl=63 time=0.189 ms

--- 192.168.1.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2049ms
rtt min/avg/max/mdev = 0.170/0.187/0.204/0.013 ms
```

Connectivity tests showed that Node1 could successfully ping Node3's underlay address (192.168.2.2) with 0% packet loss and very low latency (~0.21 ms). Likewise, Node3 could ping Node1's underlay address (192.168.1.1) with 0% packet loss and similar low latency (~0.18 ms). These results verified that bidirectional communication was fully established once the static routes were in place.

Unlike sshuttle and WireGuard, static routing did not rely on tunneling or encryption. Instead, it provided clear, transparent forwarding across subnets. While more manual to configure, this approach worked for all protocols and demonstrated how simple route entries combined with IP forwarding on Node2 can achieve cross-subnet connectivity.

Next Steps

1. Tools Comparison

I will provide practical demos of sshuttle, WireGuard, and static routing. For each method, I'll include setup commands, routing table outputs, and connectivity results. I will also compare their strengths and weaknesses in terms of configuration effort, protocol support, security, and scalability.

2. Related Work

I will carry out a literature review using Google Scholar, focusing on research that has explored sshuttle, WireGuard, and static routing. At least five references will be summarized, highlighting what previous work has achieved and how it compares with my results. This review will be written in about 1000 words to give proper coverage.

3. Intuitive Contribution

I plan to add my own improvements, either by extending the features of one of the tools, enhancing existing configurations, or integrating ideas from multiple tools and open-source projects. This may include automation scripts or hybrid methods to make the approaches more efficient.

By doing these steps, I will strengthen my project, cover theoretical and practical aspects, and add originality through my own contribution.

Appendix A – Source Code

The source code associated with the experiments and results discussed in this report is included below. In addition, the complete project code is available in the **GitHub repository**: https://github.com/static_routes_sshuttle

For convenience, the code file has also been attached separately along with this report